

Day 8 — From Boole to Physical Gates

Curriculum alignment: Stage 1 — Foundations of Computation · Boolean Logic and Physical Gates

Target time: 90–120 minutes (about 45 minutes reading and whiteboard work, 30–45 minutes prediction and derivation, and 15–30 minutes optional code observation and explanation)

We can represent OFF and ON as 0 and 1. But what must be added before those states can actually compute?

Why this lesson exists

A physical system can carry a bit:

```
continuous voltage
  ↓ threshold convention
stable LOW or HIGH
  ↓ encoding
logical 0 or 1
```

That gives us **representation**. It does not yet give us **computation**.

Suppose two switches are lying on a table. Each can be open or closed. We now have two physical states to observe, but no rule such as:

```
turn on only if both switches are closed
turn on if either switch is closed
produce the opposite of this input
```

The switches do not acquire those rules merely because we call their states `0` and `1`. To compute, we need a physical arrangement whose output changes according to a chosen rule.

That distinction is today's central problem:

```
representation: a state stands for 0 or 1
computation:   arranged devices transform input states into an output state
```

By the end, you should be able to derive NOT, AND, and OR from physical switching arrangements, write their truth tables, and explain why a useful gate must restore its output to valid voltage levels.

The path ahead is:

```
physical switch
  → logic gate
  → larger gate network
  → arithmetic and selection circuits
  → ALU
  → CPU datapath and control
```

Today we take only the first step. We will not consume XOR, NAND, De Morgan's laws, or adders before their own lessons.

1. Boole's abstraction: separate the rule from the subject

Consider three ordinary statements:

```
the machine may run if power is present AND the guard is closed
the alarm sounds if smoke is detected OR heat is excessive
the lamp is on if the control is NOT inactive
```

Their subjects differ, but their logical shapes repeat.

George Boole developed an algebra in which symbols could stand for propositions and operations could combine them. We can remove the distracting details:

```
P AND G
S OR H
NOT A
```

This was powerful because the algebra did not need to know what **P**, **G**, **S**, **H**, or **A** meant. A variable could stand for any two-way proposition. The same logical rule could describe a safety interlock, an alarm, or something not yet invented.

Boole gave us an abstract language for two-valued reasoning. He did not build a modern gate or CPU. His algebra still lived on paper.

That leaves an engineering question:

```
Can physical machinery be arranged so that its observable behavior follows the same algebra?
```

2. Shannon recognized Boolean structure in relay networks

Telephone systems already used **electromagnetic relays**. A small current through a coil created a magnetic field, which moved a metal contact. That contact opened or closed another electrical path.

```
control current
  ↓
magnetic relay coil
  ↓ moves
physical contact
  ↓
another circuit opens or closes
```

Engineers could combine many contacts to route calls and control equipment, but a large network quickly became difficult to reason about. Claude Shannon's work showed systematically that Boolean algebra could describe and simplify relay and switching circuits.

A useful convention was:

```
condition false ↔ contact open ↔ 0
condition true  ↔ contact closed ↔ 1
```

Under that convention:

- contacts in series behave like AND;
- contacts in parallel behave like OR;
- a normally closed contact provides NOT-like inversion.

Now a wiring problem could be written as an algebraic rule, checked, and implemented as a physical network.

Attribution needs care

Shannon's treatment was extraordinarily influential, but the history should not be reduced to "one person invented switching logic." Related connections between logical algebra and switching networks were developed independently in other places, and working relay circuits came from a wider engineering tradition. Shannon's distinctive contribution was a rigorous and practical synthesis that made Boolean algebra a powerful design method for relay networks.

Likewise:

```
Boole did not design a CPU
Shannon did not invent every relay circuit
a relay or transistor alone is not a computer
```

The modern machine emerged by composing ideas across mathematics, electrical engineering, materials, manufacturing, and architecture.

3. Why switching technology changed

Relays make switching visible: the contact really moves. That is also their limitation.

```
relay
  advantage: clear controlled switch, electrical isolation
  limitation: moving parts, delay, contact bounce, wear, size, noise
```

Vacuum tubes could switch or amplify electronically without moving contacts. They made much faster machines possible, but they were bulky, hot, power-hungry, and prone to failure when used in large quantities.

The transistor replaced the hot glass tube with a much smaller solid-state device. A small electrical input could control a conducting path. Transistors eventually became small and reliable enough to place huge numbers on integrated circuits.

The causal path is not merely a list of inventions:

relays made controlled switching practical
↓ but mechanical motion limited speed and reliability
vacuum tubes removed mechanical motion
↓ but size, heat, power, and failure remained severe
transistors provided compact solid-state switching
↓ integration made vast gate networks practical
modern chips compose many transistor networks

Changing the device did not change the abstract truth table. AND remains AND whether contacts, tubes, or transistors realize it. What changed was how quickly, densely, reliably, and efficiently the physical behavior could be produced.

4. A truth table is the behavioral contract

Before wiring anything, we need a precise way to state the required rule.

A **truth table** lists the output for every allowed input combination. If there are two one-bit inputs, there are:

$2^2 = 4$ input combinations

If there is one input, there are two combinations.

A truth table says what must happen, not how to build it:

truth table	→ complete input/output behavior
Boolean expression	→ compact symbolic rule
physical circuit	→ one implementation of that rule

Several different circuits may satisfy the same truth table. They compute the same Boolean function even if their component counts, delays, power use, and layouts differ.

Keep those layers separate. The table is a specification. The circuit is a physical realization. The voltages are what actually exist at runtime.

5. Derive AND from a physical path

Start with a power source, two ordinary contacts, and a lamp:

power —[A]—[B]— lamp — return

The contacts are in **series**. There is only one path through the circuit.

Work through the possibilities:

- If **A** and **B** are both open, the path is broken.
- If **A** is closed but **B** is open, the path is still broken.
- If **A** is open but **B** is closed, the path is still broken.
- Only when both are closed is the path complete.

Now record the behavior:

	A	B	Y
	0	0	0
	0	1	0
	1	0	0
	1	1	1

This is AND:

$Y = A \text{ AND } B$

The word AND is not hidden inside either contact. The behavior emerges from the **series arrangement** plus our interpretation of open/closed and off/on.

At the logic layer:

AND produces 1 only when every required input is 1.

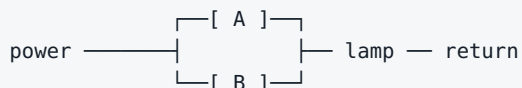
A safety interlock is a natural use:

`motor_enable = power_ready AND guard_closed`

The circuit does not understand motors or safety. It only follows the physical consequences of its arrangement.

6. Derive OR from alternate paths

Rearrange the same two contacts:



Now the contacts are in **parallel**. There are two possible paths.

- If both contacts are open, no path conducts.
- If only **A** closes, the upper path conducts.
- If only **B** closes, the lower path conducts.
- If both close, current can still reach the lamp.

The truth table is:

	A	B	Y
	0	0	0
	0	1	1
	1	0	1
	1	1	1

This is OR:

```
Y = A OR B
```

At the logic layer:

OR produces 1 when at least one input is 1.

Notice the final row. OR is inclusive: if both inputs are 1, the output is still 1.

An alarm might use:

```
alarm = smoke_detected OR heat_detected
```

Again, the physical circuit has no idea what smoke or heat means.

7. Derive NOT from a complementary action

NOT has one input and asks for the opposite output:

A	Y
0	1
1	0

```
Y = NOT A
```

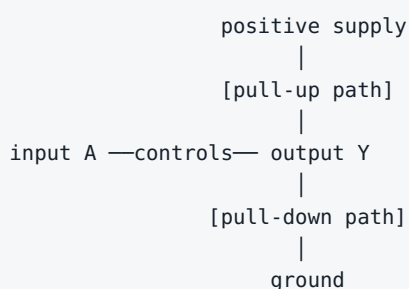
Adding another ordinary switch in series cannot create this rule. An ordinary normally open contact conducts when activated; another such contact only adds another condition for conduction.

A relay can instead use a **normally closed** contact:

```
relay coil inactive → contact closed → output active  
relay coil energized → contact opens → output inactive
```

The control action produces the complementary path state.

Modern transistor logic obtains inversion without a moving contact. A simplified CMOS inverter has a controlled pull-up path to the positive supply and a controlled pull-down path to ground:



In steady operation:

A LOW → pull-up conducts, pull-down does not → Y driven HIGH
A HIGH → pull-down conducts, pull-up does not → Y driven LOW

The exact transistor devices and their complementary behavior matter in a real schematic. For today's model, keep the essential idea: the network does not merely interrupt one passive wire. It provides one path that drives the output HIGH and another that drives it LOW.

8. Why a real gate must restore valid levels

The lamp diagrams teach topology, but they leave out an essential property of digital logic.

Real voltage is continuous. A signal may be weakened by resistance, loaded by later circuitry, disturbed by noise, or caught between rails during a transition. If every stage merely passed a slightly worse copy onward, a long chain would eventually stop representing reliable 0s and 1s.

A useful gate therefore has specified input ranges and deliberately drives its output toward valid ranges:

```
input in valid LOW range
    ↓ gate's transistor network responds
output driven toward the LOW or HIGH rail required by the function
```

For example, imagine a technology with a simplified convention:

```
0.0 V to 0.8 V → valid LOW
2.0 V to 5.0 V → valid HIGH
0.8 V to 2.0 V → not guaranteed as either
```

Those numbers are only illustrative; actual limits belong to a specific logic family and supply voltage.

Suppose an input arrives as 0.3 V. A NOT gate does not ideally output a fragile, proportionally transformed middle voltage. It recognizes the input as LOW and actively drives its output toward a valid HIGH level. The next gate receives a renewed signal.

This is **restoration**:

```
imperfect but valid input level
    ↓
gate action
    ↓
fresh output near a valid logic rail
```

Restoration is one reason gates can be composed. It gives later stages a clean logical state rather than asking them to interpret an indefinitely degrading analog trace.

It does not make physics disappear. Real gates still have:

- propagation delay;
- limits on how many inputs an output can drive;
- power consumption;
- transition times;

- noise margins;
- and invalid or uncertain behavior outside specified conditions.

Truth tables describe settled logical behavior. Device specifications tell us whether a physical circuit can produce that behavior reliably and quickly enough.

9. Prediction exercise: arrange first, name second

Do this on paper before reading the discussion beneath each prompt.

Prediction A — series path

Three contacts **A**, **B**, and **C** are connected in series:

```
power —[ A ]—[ B ]—[ C ]— output
```

1. How many input combinations exist?
2. For which combinations is the output active?
3. Write one sentence describing the logical rule.

Prediction B — parallel paths

Three contacts are connected in parallel:

```

power ———┐
           ├──[ A ]──┐
           ├──[ B ]──┐ output
           └──[ C ]──┘

```

1. For which single input combination is the output inactive?
2. If **A=1**, does changing **B** alter the settled output?
3. Write the logical rule without using a gate symbol.

Prediction C — mixed physical arrangement

```

power ———┐
           ├──[ A ]──[ B ]──┐
           └──[ C ]──┘      output

```

Predict all eight rows. Use an intermediate statement:

```

upper path conducts when ...
lower path conducts when ...
output is active when ...

```

Do not introduce any gate beyond AND and OR. The goal is to move between topology, words, and a truth table.

Prediction D — restoration

A valid but noise-affected LOW reaches a NOT gate.

1. Should the output be another weak LOW, a valid HIGH, or an unspecified analog copy?

2. What physical feature of an inverter makes the desired output possible?
3. Why is the truth table alone insufficient to answer whether ten thousand such stages will meet a timing requirement?

10. Optional Python observation

Software can enumerate the same abstract input/output rules. It does not turn your CPU into the relay diagrams above; the running CPU already contains physical gates that execute the program.

Predict every printed row first:

```
python3 - <<'PY'
def bit_not(a):
    return 1 - a

print("A | NOT")
for a in (0, 1):
    print(a, "|", bit_not(a))

print("\nA B | AND OR")
for a in (0, 1):
    for b in (0, 1):
        print(a, b, "| ", a & b, " ", a | b)
PY
```

Then explain the layers:

Python source	describes requested operations
language implementation	and CPU execute them
transistor networks	perform physical state transitions
printed rows	let us observe the abstract truth tables

The program is an observation tool, not a transistor-level model.

11. Optional C observation

Create a temporary file outside the repository if you want to run this:

```
#include <stdio.h>

int main(void)
{
    puts("A | NOT");
    for (int a = 0; a <= 1; ++a)
        printf("%d | %d\n", a, !a);

    puts("\nA B | AND OR");
    for (int a = 0; a <= 1; ++a) {
        for (int b = 0; b <= 1; ++b)
            printf("%d %d | %d %d\n", a, b, a && b, a || b);
    }

    return 0;
}
```

Before running it, predict the output. Then build with:

```
gcc -std=c17 -Wall -Wextra -O0 -g /tmp/logic-observe.c \
-o /tmp/logic-observe
/tmp/logic-observe
```

For inputs restricted to 0 and 1, C's logical operators produce the rows we want. Do not overgeneralize:

- `!`, `&&`, and `||` are logical operators;
- `&&` and `||` short-circuit;
- `&` and `|` are bitwise integer operators;
- truth-table agreement on one-bit inputs does not make their C semantics interchangeable.

We will return to bitwise behavior in its own software context.

12. From these gates toward a CPU

NOT, AND, and OR are modest transformations, but they change the nature of the machine. We no longer have bits that merely sit for interpretation. We have physical networks that map input bits to output bits.

Larger networks can be built by feeding gate outputs into later gate inputs:

```
input bits
  ↓
NOT / AND / OR networks
  ↓
larger combinational functions
  ↓
arithmetic, comparison, and selection circuits
  ↓
ALU
  ↓
registers + control + timing
  ↓
CPU
```

Do not skip the missing engineering.

An ALU requires larger functions and a way to select among them. A CPU also needs storage, controlled sequencing, and instruction interpretation. Today's gates are necessary building blocks, not a complete processor.

The important continuity is:

Boole: express a two-valued rule independently of its subject
Shannon: use that algebra to analyze and design switching networks
hardware: arrange physical devices so settled outputs match the rule
CPU: compose enormous numbers of such transformations with state and control

At no point does a switch “understand” logic. The logical function is our description of reliable physical behavior.

13. Mastery questions

Close the lesson before answering.

Explain

1. Why is the ability to represent 0 and 1 not yet computation?
2. What did Boole abstract away, and why was that useful?
3. What practical design connection did Shannon make with relay circuits?
4. Why is “Shannon alone invented switching logic” historically too simple?
5. What limitation motivated each transition from relays to vacuum tubes to transistors?
6. Where does AND exist: in the word, the truth table, the switch topology, or the voltage behavior? Explain the layers rather than choosing only one.

Derive

1. Draw a two-input AND using contacts. Derive all four truth-table rows from continuity of the electrical path.
2. Draw a two-input OR using contacts. Why does the 1,1 row still produce 1?
3. Why cannot a chain of ordinary normally open contacts implement NOT?
4. Describe how a normally closed relay contact and a CMOS inverter each create inversion, while noting that their physics differs.

Connect

1. Why must a gate actively restore an output toward valid voltage levels?
2. What physical facts are hidden by a truth table?
3. Explain how three small gate types can point toward an ALU without claiming that we have already built one.
4. A circuit produces the right truth table when tested slowly but fails at high speed. Is the Boolean rule wrong? What other layer must be examined?

Mastery prompt

Answer in one uninterrupted explanation:

How can a physical circuit implement a logical rule without understanding what its bits mean?

A strong answer connects voltage ranges, switching devices, physical topology, restored outputs, truth tables, and interpretation.

Why? notebook

Write short answers in your own words:

1. Why does naming two physical states **0** and **1** create a representation but not a transformation?
2. Why was it useful for Boole's algebra to ignore what each proposition was about?
3. Why did telephone relay networks create a practical home for an abstract algebra?
4. Why is series wiring naturally described as AND and parallel wiring as OR?
5. Why does inversion require a complementary action rather than merely one more ordinary series contact?
6. Why can a relay circuit and a transistor circuit implement the same truth table despite using different physics?
7. Why must output restoration be part of the story when gates are composed?
8. Why does a correct truth table say nothing by itself about delay, heat, loading, or reliability?
9. Why are gates a route toward an ALU and CPU but not yet either one?

Completion check

You are ready for the next gate lesson when you can:

- distinguish representation from computation;
- explain the Boole-to-Shannon connection without a single-inventor myth;
- give the engineering reason for relays → vacuum tubes → transistors;
- derive NOT, AND, and OR rather than recite their tables;
- move between a physical arrangement, a Boolean rule, and a truth table;
- explain restoration in terms of valid voltage ranges;
- and connect gate composition toward an ALU and CPU without jumping ahead.

If one item is weak, redraw one concrete circuit and explain every layer aloud.

References used selectively

Local reading

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 10–12, from “Logic and Switches” through the opening of “A Binary Adding

Machine” (local PDF pages 98–145): [source-materials/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf](#)

- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 3, §§3.1–3.3 and §3.3.5, on transistor switches, gates, combinational logic, and logical completeness (book pages 51–64): [source-materials/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf](#)

Historical and primary context

- University of St Andrews, MacTutor, “George Boole,” on the development of algebraic logic: <https://mathshistory.st-andrews.ac.uk/Biographies/Boole/>
- MIT Libraries, Claude E. Shannon, *A Symbolic Analysis of Relay and Switching Circuits*: <https://dspace.mit.edu/handle/1721.1/11173>
- Computer History Museum, “Inventing the Transistor,” on Bell Labs' search for a semiconductor replacement for vacuum tubes and the first working transistor: <https://www.computerhistory.org/revolution/digital-logic/12/273>
- NAND2Tetris, Project 1, “Boolean Logic,” for the later gate-construction path from elementary gates toward a CPU: <https://www.nand2tetris.org/project01>

Next bridge: extend these basic physical and logical ideas into a wider gate vocabulary and composed circuits, while continuing to separate abstract behavior from electrical implementation.